

Generative AI from first principles

Madhusudhan Reddy Pittu

Lecture 1: (Artificial) Intelligence, Learning, Language, and Meaning

1. Short timeline highlighting language and its modeling,
course description: what we will study and build
2. Basic AI/ML/DL terminology
3. Understanding
 - Intelligence
 - Learning
 - Language
 - Meaning
4. Hands-on: an autograd engine

From **Computation** to **Language Models**

Humans stand out among all other in one respect above all:

1. we build tools that make our lives better.
2. we pass those tools on.

This is taken even further by the ability to transform information: Computation.



control of fire



the wheel



steam power

Physical



writing



printing



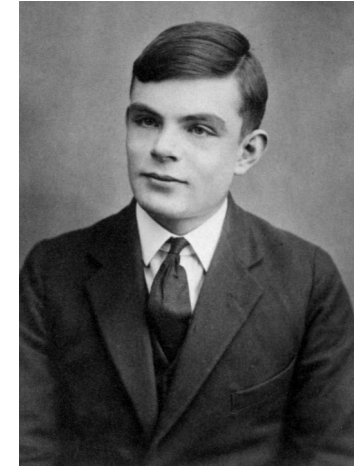
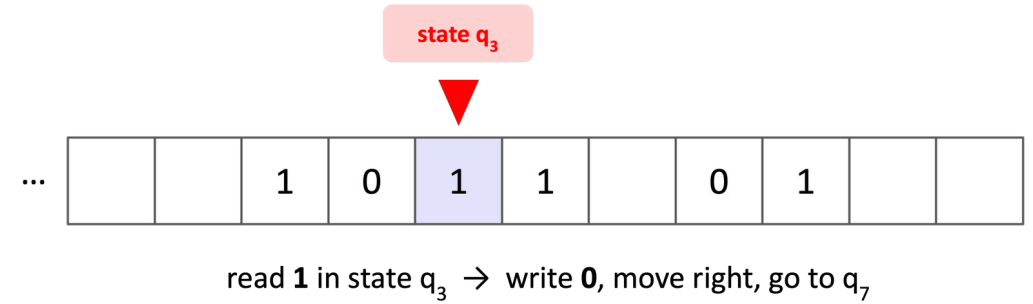
the telegraph

Informational

From **Computation** to **Language Models**

This is taken even further by the ability to transform information: Computation.

1936: computation becomes a mathematical object



Alan Turing

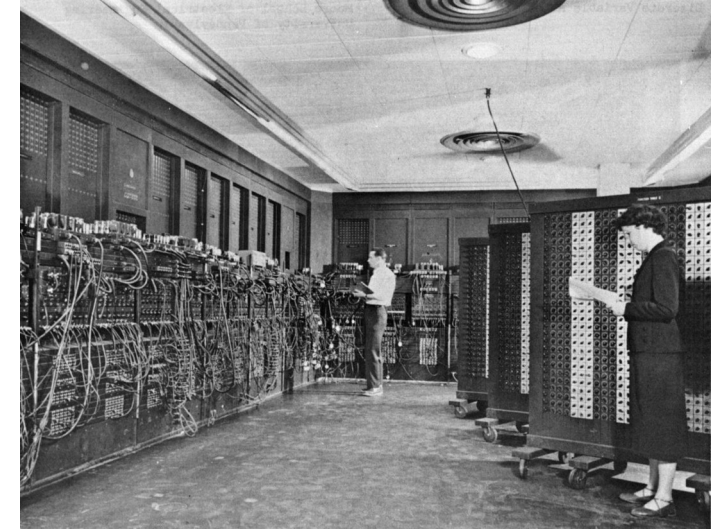
A finite table of rules, reading and writing symbols on a tape, captures what it means to compute; one universal machine can run any other.

From **Computation** to **Language Models**

This is taken even further by the ability to transform information: Computation.

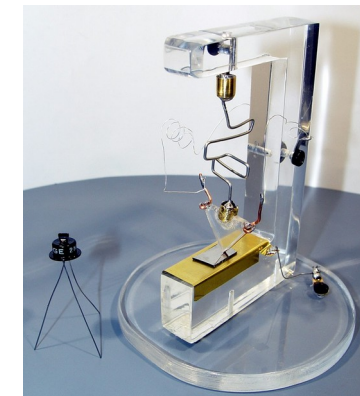
1936: computation becomes a mathematical object

1946–1947: computers become physical objects



ENIAC, announced 1946

ENIAC: about 18,000 vacuum tubes, reprogrammed by rewiring. The transistor (Bell Labs, 1947) made computers small, cheap, and reliable.



the first transistor (replica), 1947

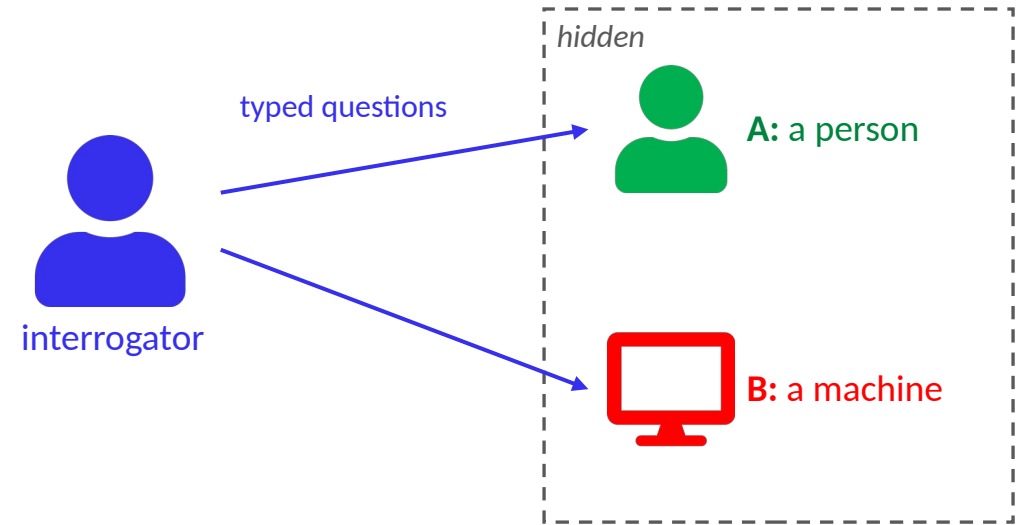
From **Computation** to Language Models

This is taken even further by the ability to transform information: Computation.

1936: computation becomes a mathematical object

1946–1947: computers become physical objects

1950: the imitation game



Which one is the machine?

Typed conversation only. If the interrogator cannot reliably tell which is the machine, the machine passes.

From **Computation** to **Language Models**

This is taken even further by the ability to transform information: Computation.

1936: computation becomes a mathematical object

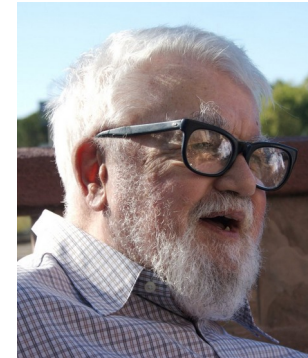
1946–1947: computers become physical objects

1950: the imitation game

1955–1956: artificial intelligence gets a name

“Every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it.”

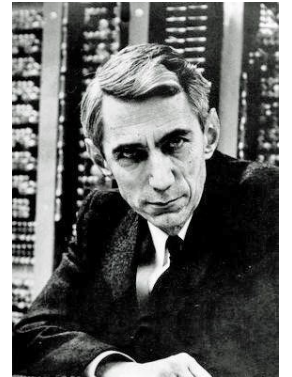
the Dartmouth proposal, 1955



John McCarthy



Marvin Minsky



Claude Shannon

with Nathaniel Rochester of IBM, the fourth author

The proposal (August 1955) coined the name “artificial intelligence”; the summer workshop of 1956 started the field.

From **Computation** to **Language Models**

This is taken even further by the ability to transform information: Computation.

1936: computation becomes a mathematical object

1946–1947: computers become physical objects

1950: the imitation game

1955–1956: artificial intelligence gets a name

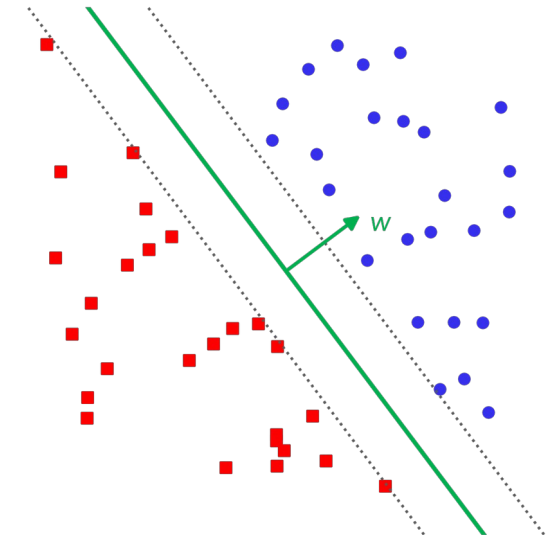
1958: the perceptron learns from examples

ML



The Mark I Perceptron at work ([film clip](#))

Rosenblatt's perceptron nudges its weights after each mistake, until a line separates the two classes: an early landmark of machine learning.



From **Computation** to **Language Models**

This is taken even further by the ability to transform information: Computation.

1936: computation becomes a mathematical object

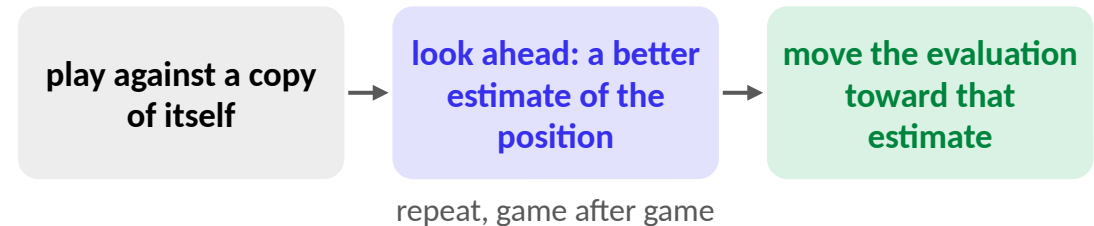
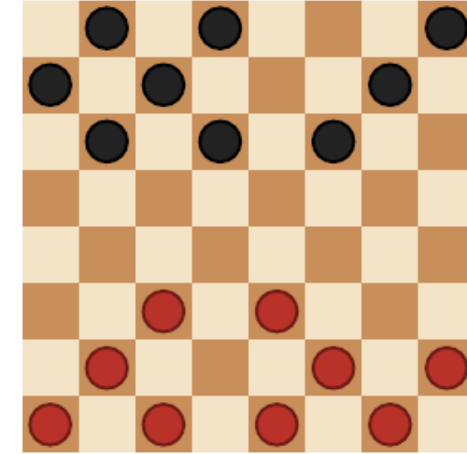
1946–1947: computers become physical objects

1950: the imitation game

1955–1956: artificial intelligence gets a name

1958: the perceptron learns from examples ML

1959: Samuel's checkers player learns from self-play RL



Arthur Samuel's program played checkers against itself and moved its evaluation of positions toward what looking ahead revealed: an early landmark of reinforcement learning. His paper's title is an early use of the phrase “machine learning”.

From **Computation** to Language Models

This is taken even further by the ability to transform information: Computation.

1936: computation becomes a mathematical object

1946–1947: computers become physical objects

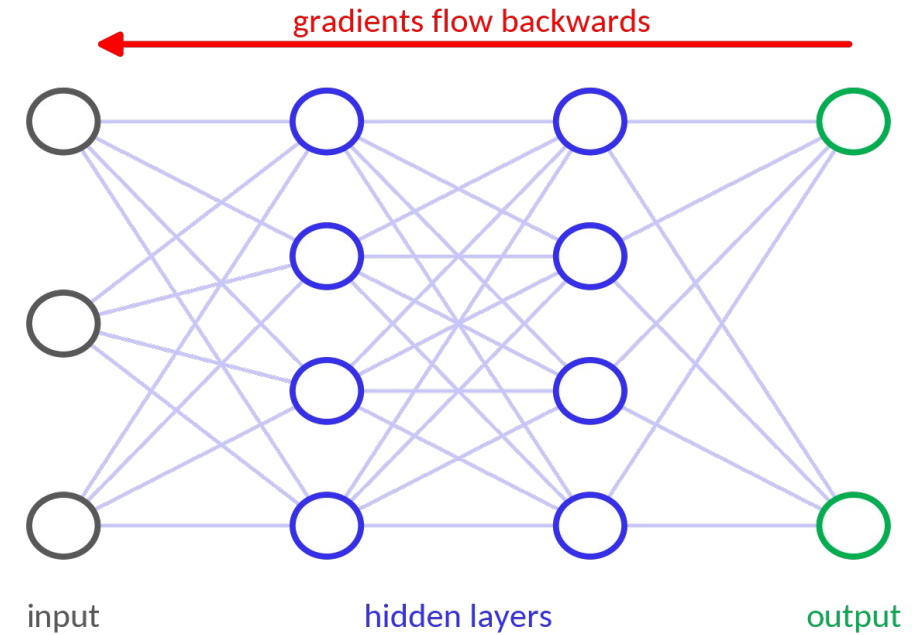
1950: the imitation game

1955–1956: artificial intelligence gets a name

1958: the perceptron learns from examples ML

1959: Samuel's checkers player learns from self-play RL

1986: networks learn their own features DL



The chain rule, run backwards, gives every weight's gradient in one pass, so hidden layers learn their own features: an early landmark of deep learning.



David Rumelhart



Geoffrey Hinton

From **Computation** to **Language Models**

This is taken even further by the ability to transform information: Computation.

1936: computation becomes a mathematical object

1946–1947: computers become physical objects

1950: the imitation game

1955–1956: artificial intelligence gets a name

1958: the perceptron learns from examples ML

1959: Samuel's checkers player learns from self-play RL

1986: networks learn their own features DL

1997: Deep Blue defeats Kasparov AI

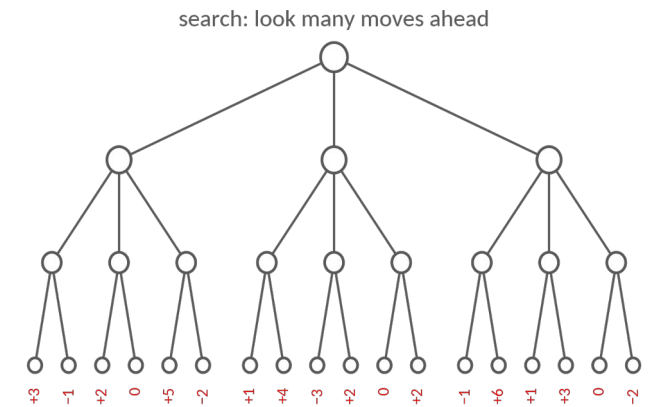


Kasparov against Deep Blue, 1996



Deep Blue

Deep Blue searched up to 200 million positions a second and scored them with an evaluation function written by chess experts: a landmark of classical, search-based AI.



evaluation written by human experts

From **Computation** to Language Models

This is taken even further by the ability to transform information: Computation.

1936: computation becomes a mathematical object

1946–1947: computers become physical objects

1950: the imitation game

1955–1956: artificial intelligence gets a name

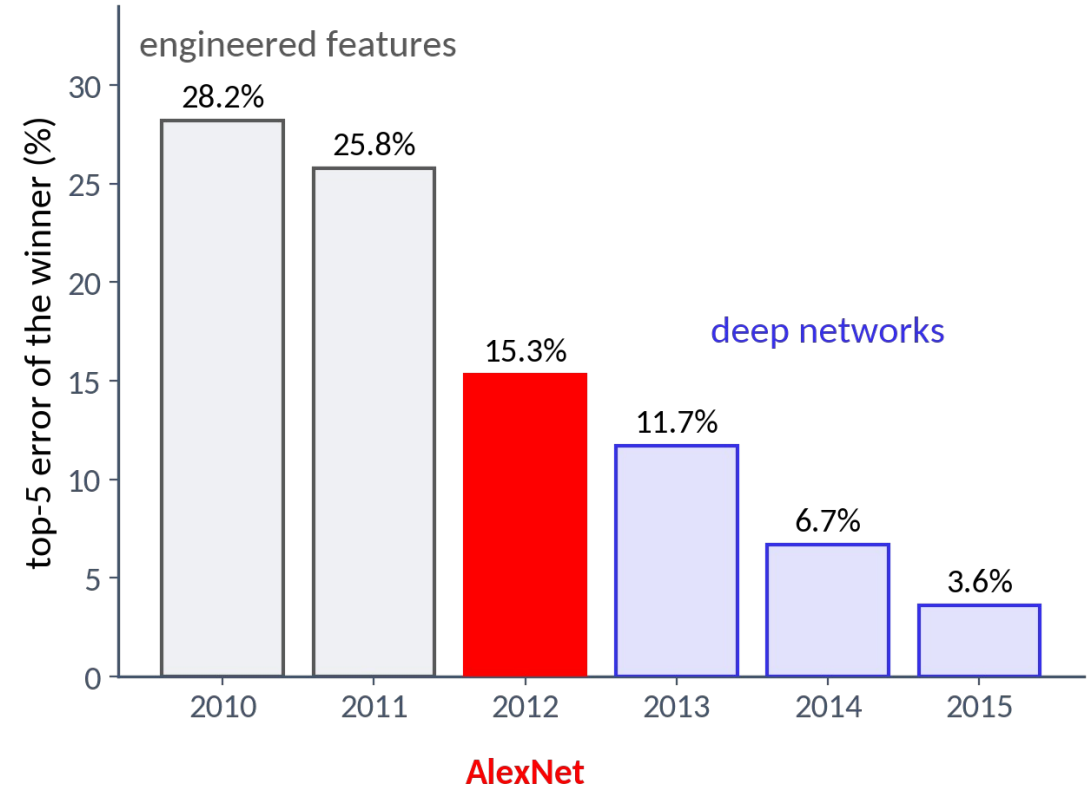
1958: the perceptron learns from examples ML

1959: Samuel's checkers player learns from self-play RL

1986: networks learn their own features DL

1997: Deep Blue defeats Kasparov AI

2012: AlexNet wins ImageNet



ImageNet: 1.2 million labeled photos in 1,000 classes. AlexNet, a deep convolutional network trained on two GPUs, learned its own features and cut the winning error from 25.8% to 15.3%.

From **Computation** to Language Models

This is taken even further by the ability to transform information: Computation.

1936: computation becomes a mathematical object

1946–1947: computers become physical objects

1950: the imitation game

1955–1956: artificial intelligence gets a name

1958: the perceptron learns from examples ML

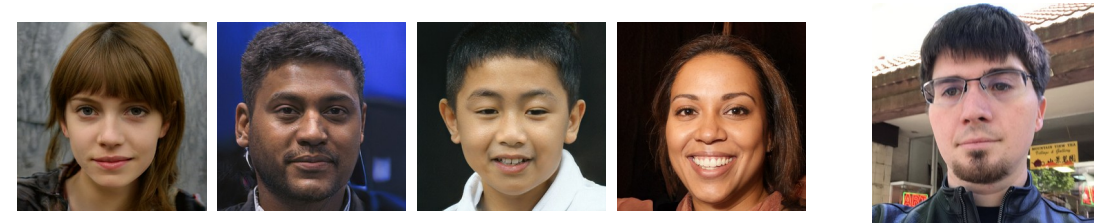
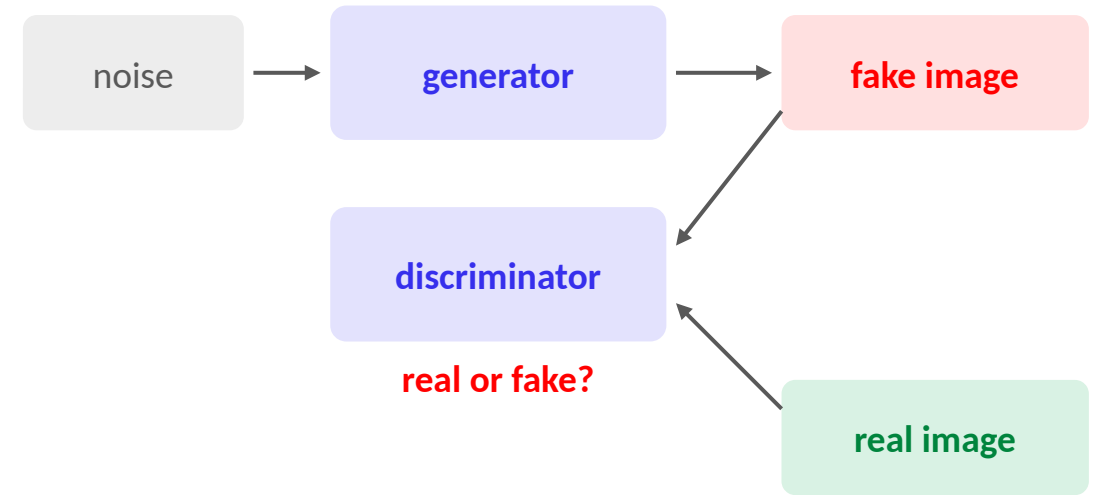
1959: Samuel's checkers player learns from self-play RL

1986: networks learn their own features DL

1997: Deep Blue defeats Kasparov AI

2012: AlexNet wins ImageNet

2014: generation before ChatGPT



none of these people exist (StyleGAN, 2019–2020)



Ian Goodfellow

A generator turns random noise into images while a discriminator learns to tell them from real ones; each improves against the other (Goodfellow et al., 2014). The faces come from a later descendant.

From **Computation** to **Language Models**

This is taken even further by the ability to transform information: Computation.

1936: computation becomes a mathematical object

1946–1947: computers become physical objects

1950: the imitation game

1955–1956: artificial intelligence gets a name

1958: the perceptron learns from examples ML

1959: Samuel's checkers player learns from self-play RL

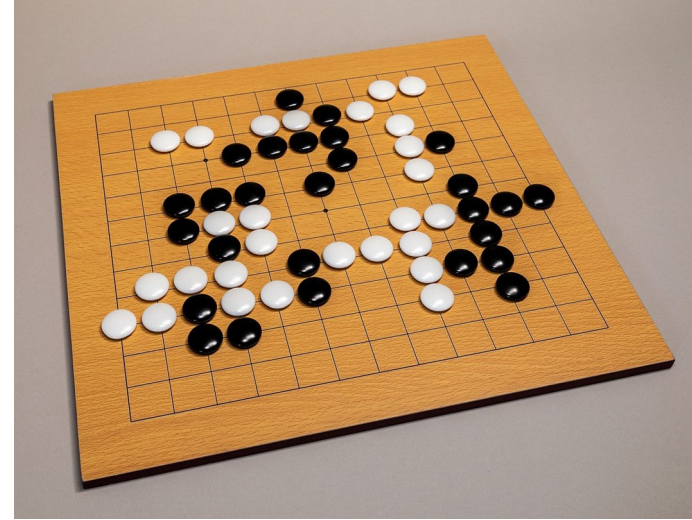
1986: networks learn their own features DL

1997: Deep Blue defeats Kasparov AI

2012: AlexNet wins ImageNet

2014: generation before ChatGPT

2016: AlphaGo defeats Lee Sedol, 4–1



a game of Go



Lee Sedol

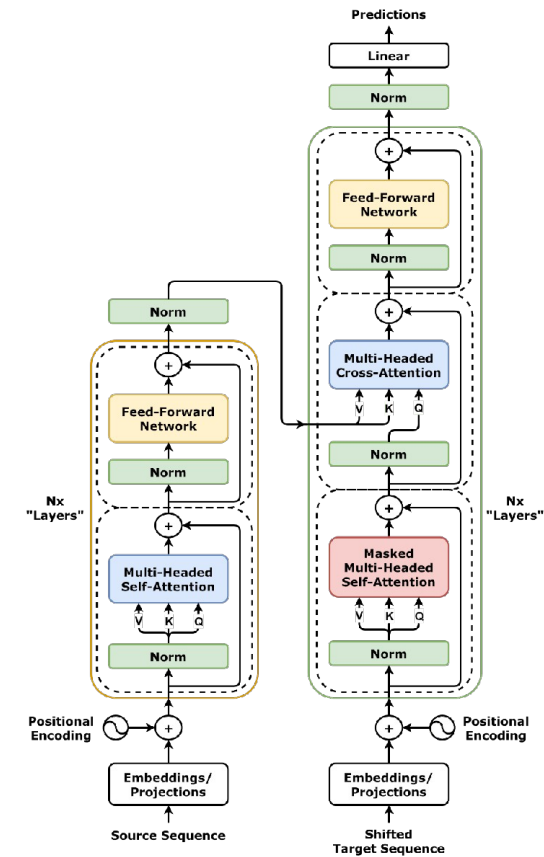
Networks learned from expert games and from self-play which moves to consider and who is winning; a tree search combined them. AlphaGo won the match 4–1.

From Computation to Language Models

This is taken even further by the ability to transform information: Computation.

Language Models

2017: the Transformer



Designed for machine translation: the encoder (left) reads a sentence and the decoder (right) writes its translation. Every position reads from the others through attention. The decoder alone, trained to predict the next token, is a language model; we build one in Week 3.

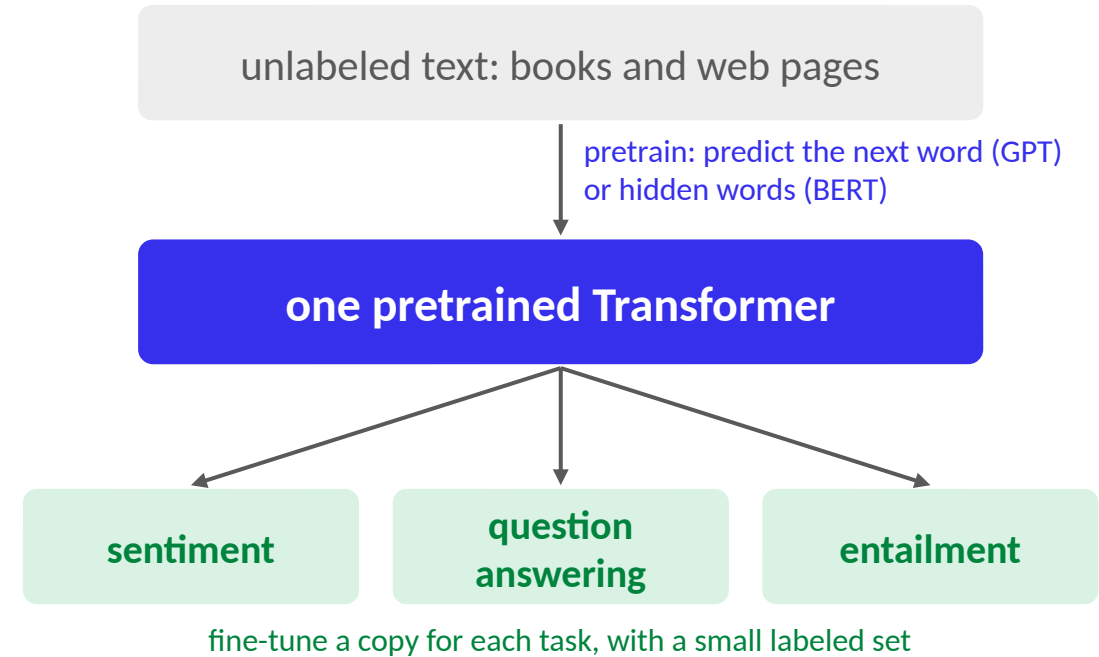
From **Computation** to **Language Models**

This is taken even further by the ability to transform information: Computation.

Language Models

2017: the Transformer

2018: GPT and BERT: pretrain, then fine-tune



Pretraining on unlabeled text gives one network that already knows much about language; fine-tuning a copy on each task beat models built for that task alone. GPT reads left to right; BERT reads in both directions.

From **Computation** to **Language Models**

This is taken even further by the ability to transform information: Computation.

Language Models

2017: the Transformer

2018: GPT and BERT: pretrain, then fine-tune

2020: GPT-3: a task from a few examples

prompt

Review: The plot dragged; the end made no sense.
Sentiment: negative

Review: A warm, funny film with a great cast.
Sentiment: positive

Review: I'd happily watch it again tomorrow.
Sentiment:

the model continues: **positive**

GPT-3 has 175 billion parameters. The task is given in the prompt by a few examples, and nothing is retrained: the weights stay fixed. Ask the model what comes next, then reveal.

From Computation to Language Models

This is taken even further by the ability to transform information: Computation.

Language Models

2017: the Transformer

2018: GPT and BERT: pretrain, then fine-tune

2020: GPT-3: a task from a few examples

2022: ChatGPT: an assistant you can talk to

Write a Python function that returns the n-th Fibonacci number.

```
def fib(n):  
    return n if n < 2 else fib(n-1) + fib(n-2)
```

It is slow for n = 40. Make it fast.

```
def fib(n):  
    a, b = 0, 1  
    for _ in range(n):  
        a, b = b, a + b  
    return a
```

an illustrative exchange

Post-training on demonstrations and human preferences, plus a chat interface, turned a language model into an assistant that follows instructions (announced 30 November 2022).

From **Computation** to **Language Models**

This is taken even further by the ability to transform information: Computation.

Language Models

2017: the Transformer

2018: GPT and BERT: pretrain, then fine-tune

2020: GPT-3: a task from a few examples

2022: ChatGPT: an assistant you can talk to

2023–2024: GPT-4 and GPT-4o: images and audio in



What flower is this?

An iris: most likely Iris versicolor,
the blue flag.

an illustrative exchange

GPT-4 (2023) accepted images as well as text; GPT-4o (2024) took text, audio, and images in one model. The same flower returns in the terminology.

From Computation to Language Models

This is taken even further by the ability to transform information: Computation.

Language Models

2017: the Transformer

2018: GPT and BERT: pretrain, then fine-tune

2020: GPT-3: a task from a few examples

2022: ChatGPT: an assistant you can talk to

2023–2024: GPT-4 and GPT-4o: images and audio in

2024–2025: o1 and R1: reasoning before answering

Find all x in $\{0, 1, \dots, 34\}$ with $x^2 \equiv 1 \pmod{35}$.

thinking ...

$35 = 5 \cdot 7$, so $x^2 \equiv 1$ modulo 5 and modulo 7.

$x \equiv \pm 1 \pmod{5}$ and $x \equiv \pm 1 \pmod{7}$: four cases.

Combine them: 1, 6, 29, 34. Check: $6^2 = 36 = 35 + 1$.

answer: 1, 6, 29, 34



checked ✓ → reward

reinforcement learning on many such problems updates the model

Rewarding answers that check out taught models to think at length before answering (OpenAI o1, 2024; DeepSeek-R1, 2025): more computation at inference, more answers right.

From **Computation** to Language Models

This is taken even further by the ability to transform information: Computation.

Language Models

2017: the Transformer

2018: GPT and BERT: pretrain, then fine-tune

2020: GPT-3: a task from a few examples

2022: ChatGPT: an assistant you can talk to

2023–2024: GPT-4 and GPT-4o: images and audio in

2024–2025: o1 and R1: reasoning before answering

2025: gold-medal standard at the IMO



35/42



5 of 6 problems, 7 points each; the gold cutoff was 35

An advanced version of Gemini Deep Think wrote natural-language proofs within the 4.5-hour limit, graded by the IMO coordinators (July 2025).

Gemini Deep Think at gold-medal level: [the post on r/singularity](#)

From **Computation** to **Language Models**

This is taken even further by the ability to transform information: Computation.

Language Models

2017: the Transformer

2018: GPT and BERT: pretrain, then fine-tune

2020: GPT-3: a task from a few examples

2022: ChatGPT: an assistant you can talk to

2023–2024: GPT-4 and GPT-4o: images and audio in

2024–2025: o1 and R1: reasoning before answering

2025: gold-medal standard at the IMO

2025–2026: open Erdős problems resolved



Paul Erdős with a young Terence Tao

Erdős left over a thousand problems. Language models, sometimes with the Lean proof assistant, began resolving them; Problem 728 was settled in January 2026 with a Lean-checked proof. Tao's tracker records each contribution, from finding the literature to full solutions, and cautions that some problems had simply been overlooked.

From **Computation** to **Language Models**

This is taken even further by the ability to transform information: Computation.

Language Models

2017: the Transformer

2018: GPT and BERT: pretrain, then fine-tune

2020: GPT-3: a task from a few examples

2022: ChatGPT: an assistant you can talk to

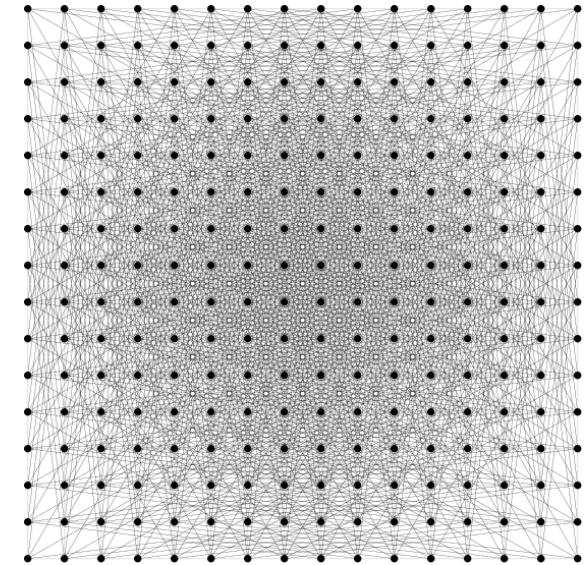
2023–2024: GPT-4 and GPT-4o: images and audio in

2024–2025: o1 and R1: reasoning before answering

2025: gold-medal standard at the IMO

2025–2026: open Erdős problems resolved

2026: the unit distance conjecture disproved



a rescaled square grid: the old record, now beaten

$$\text{Erdős (1946): } u(n) \leq n^{1+o(1)} ? \quad 2026: u(n) \geq n^{1+\delta}$$

An internal OpenAI reasoning model built point sets with far more unit distances than Erdős believed possible, using algebraic number theory. Noga Alon, Tim Gowers, and others checked it and wrote a companion paper (May 2026).

From Computation to Language Models

This is taken even further by the ability to transform information: Computation.

Language Models

2017: the Transformer

2018: GPT and BERT: pretrain, then fine-tune

2020: GPT-3: a task from a few examples

2022: ChatGPT: an assistant you can talk to

2023–2024: GPT-4 and GPT-4o: images and audio in

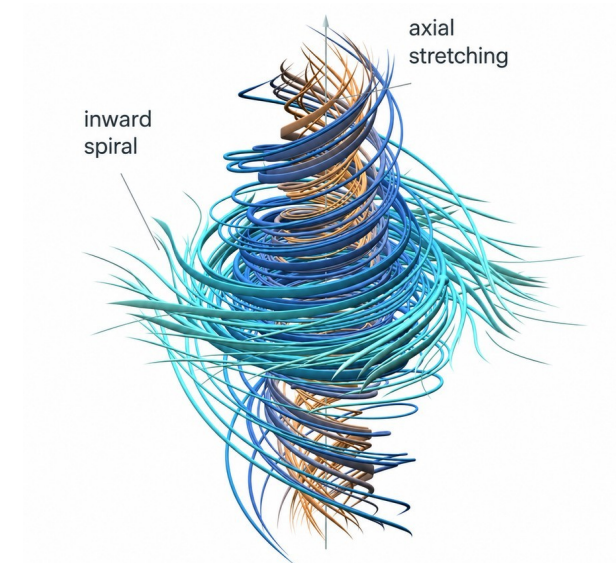
2024–2025: o1 and R1: reasoning before answering

2025: gold-medal standard at the IMO

2025–2026: open Erdős problems resolved

2026: the unit distance conjecture disproved

2026: Navier–Stokes blowup, claimed



a swirling flow, spiraling inward and stretching along its axis

About 10,000 coordinated agents produced a Lean-checked proof that a fluid at rest, driven by a smooth force, can blow up in finite time in 3D. The Clay Institute says the problem “has apparently been settled” and is reviewing; priority and the role of the forcing are debated.

Lecture 1: (Artificial) Intelligence, Learning, Language, and Meaning

1. Short timeline highlighting language and its modeling,
course description: what we will study and build
2. Basic AI/ML/DL terminology
3. Understanding
 - Intelligence
 - Learning
 - Language
 - Meaning
4. Hands-on: an autograd engine

The Language Model

A **language model** is a **generative model of text**: a **probability distribution over strings of tokens** (characters, words, or pieces of words).

It predicts **one token at a time**: each factor is a distribution over the vocabulary, given everything before it.

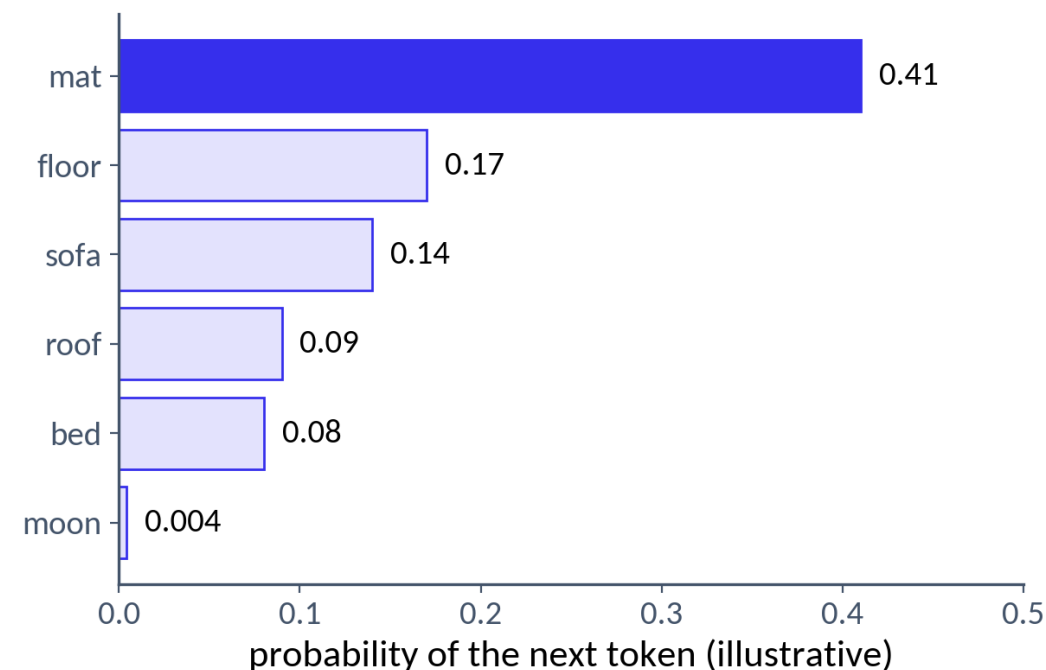
To generate text: sample a token, append it, repeat.

Not a new idea. The product is the chain rule of probability, and Shannon generated English this way in 1948.

The main innovation is how each conditional is computed and fitted.

$$q_{\theta}(x_1 x_2 \cdots x_T \text{ EOS}) = \prod_{t=1}^{T+1} q_{\theta}(x_t \mid x_{<t})$$

the cat sat on the ____



Course Plan

week	lecture	we build
Week 1	(Artificial) Intelligence, Learning, Language, and Meaning	an autograd engine
Week 2	Language Modeling: From Counts to Neural Networks	count models, word2vec, a neural LM
Week 3	Attention and Transformers: Computation and Limits	a decoder-only Transformer
Week 4	Pretraining: Data, Optimization, and Scale	a tokenizer, training loop, scaling sweep
Week 5	From Base Models to Assistants	a chatbot, by supervised fine-tuning
Week 6	Reinforcement Learning for Sequence Generation	policy gradients; rewards for the chatbot
Week 7	Post-Training: Demonstrations, Preferences, Rewards	a reward model, DPO, and GRPO
Week 8	Inference: Decoding, Test-Time Compute, Serving, Evaluation	samplers, a KV cache, speculative decoding
Week 9*	Alignment and Language Model Agents	a tool-use loop

Every model trains on a laptop in minutes to an hour. Week 9 is optional.

Lecture 1: (Artificial) Intelligence, Learning, Language, and Meaning

1. Short timeline highlighting language and its modeling, course description: what we will study and build
2. Basic AI/ML/DL terminology
3. Understanding
 - Intelligence
 - Learning
 - Language
 - Meaning
4. Hands-on: an autograd engine

Basic AI/ML/DL Terminology

Artificial intelligence

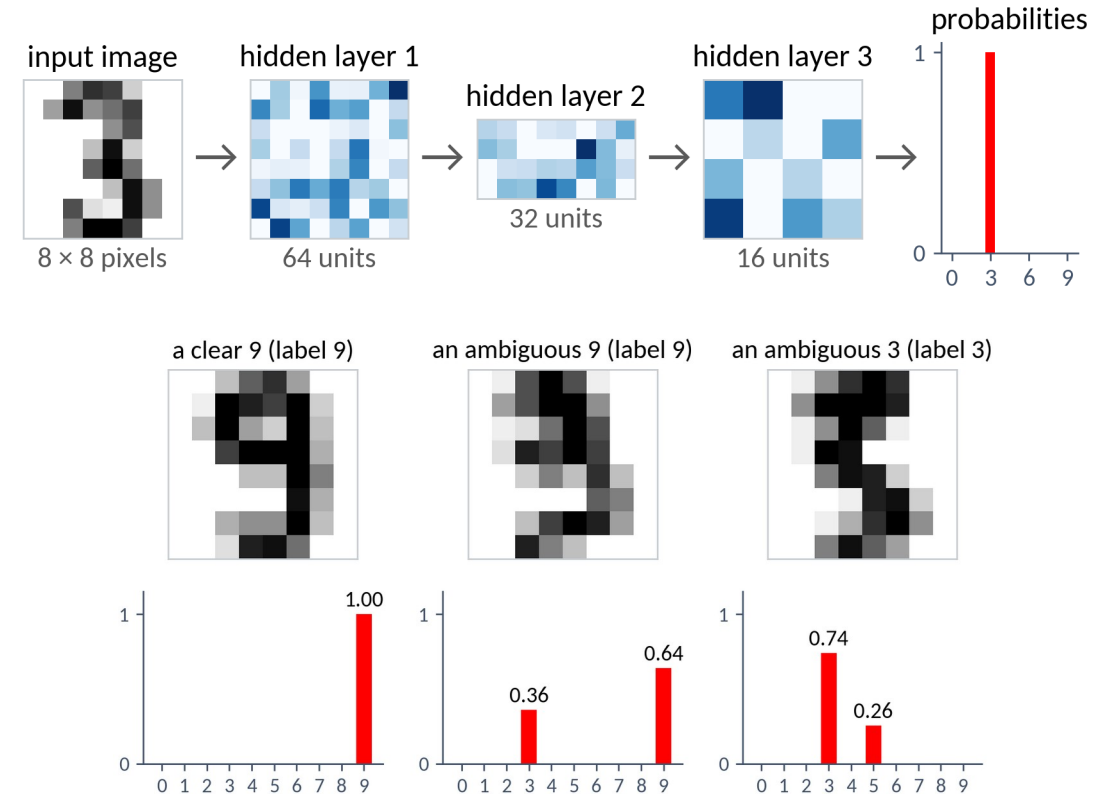
Computational systems that perform tasks associated with human intelligence: perception, reasoning, language, planning, action.
Methods: search, logic, probabilistic reasoning, learning.

Machine learning

Algorithms that learn from data and generalize to unseen data, instead of having every rule programmed. Foundations: statistics and optimization.

Deep learning

Machine learning with multilayer neural networks: many trainable transformations, composed, learn intermediate representations (features) along with the prediction.



A held-out 8×8 digit passes through three hidden layers, whose activations are learned features. The output is a probability distribution; an ambiguous digit splits it between two classes.

Basic AI/ML/DL Terminology

Learning from experience

A program learns from experience **E** with respect to a class of tasks **T** and a performance measure **P** if its performance on tasks in T, as measured by P, improves with E.

Tom Mitchell, Machine Learning (1997)

	iris flowers	a language model
T task	name a flower's species	predict the next token of text
E experience	150 labeled flowers	a corpus of documents
P performance	fraction named correctly	average log probability of held-out text

Basic **AI**/**ML**/**DL** Terminology

Describing an ML system

dimension	question	examples
Learning paradigm	What feedback is available?	supervised, unsupervised, or reinforcement learning
Learning setting	When does learning occur?	offline or online learning
Task	What result is required?	a class label, a number, a grouping, or a distribution
Model and algorithm	What is represented, and how is it learned?	a linear predictor fitted by gradient descent
Learning, prediction, and action	What is the system doing at a given moment?	updating a model, computing an output, or acting on it

The descriptions apply together: a classifier can be trained by supervised learning on an offline dataset, then used to predict as new inputs arrive. The next slides take the dimensions one at a time.

Basic **AI**/**ML**/**DL** Terminology

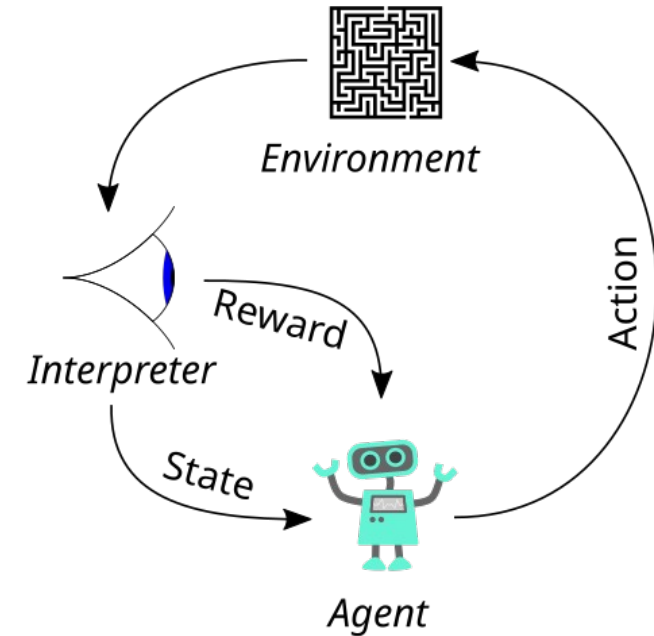
Learning paradigms: where the learning signal comes from

Supervised: each input comes with a target. Learn to predict the target for new inputs.

Unsupervised: inputs only. Find structure: groups, a density, a representation.

Self-supervised: targets constructed from the data itself, such as the next word.

Reinforcement: act and receive rewards; the actions change what is seen next.



the agent-environment loop (Foundations 1)

The agent's actions change what it observes next, so the data depend on the behavior being learned. A reward says how good the action was, not which action was best, and may arrive long after.

One Transformer: **self-supervised** pretraining → **supervised** fine-tuning → **rewards**.

Basic **AI**/**ML**/**DL** Terminology

Online and offline learning

Offline: fit on a dataset collected beforehand; the deployed model stays fixed between training runs.

Online: update the model as examples arrive, interleaving prediction and updates.

Offline learning	Online learning
fits a model to a previously collected dataset	updates a model as further examples arrive
keeps the deployed model fixed between training runs	interleaves prediction and model updates
can revisit stored examples over many passes	often processes a stream with limited storage

Not about the internet. Mini-batches do not make training online. Reinforcement learning can be offline too: learning a policy from a fixed log of another policy's actions.

Basic **AI**/**ML**/**DL** Terminology

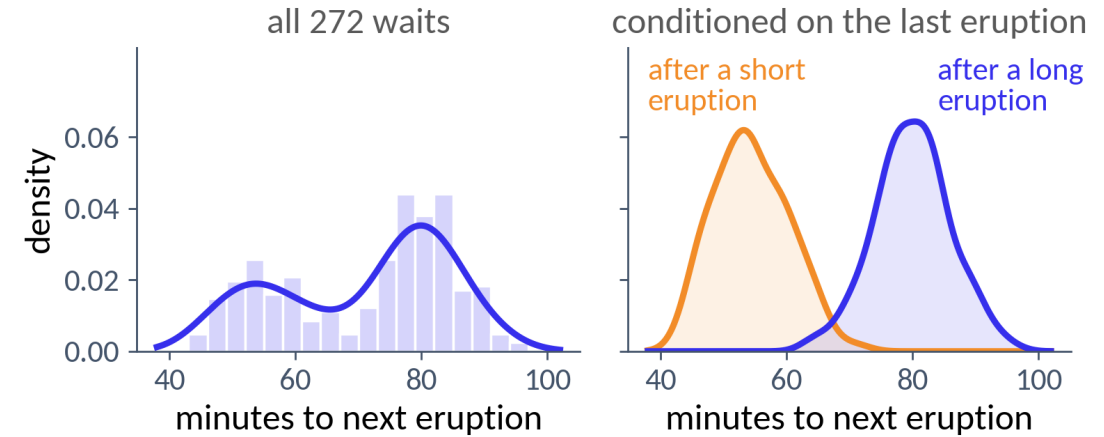
Tasks: what result is required

Classification: assign a category, or class probabilities.

Regression: predict a number.

Density estimation: estimate how the observations are distributed.

A language model is a density estimator for text;
generating text is sampling from it.



Old Faithful: 272 waits between eruptions. The density has two peaks, near 54 and 80 minutes, and the mean, 71, is rarely what happens. Conditioning on the last eruption separates the two.

Basic AI/ML/DL Terminology

Models, parameters, and architectures

Model: represents a relationship or a distribution.

Model class: the family considered; training picks one member.

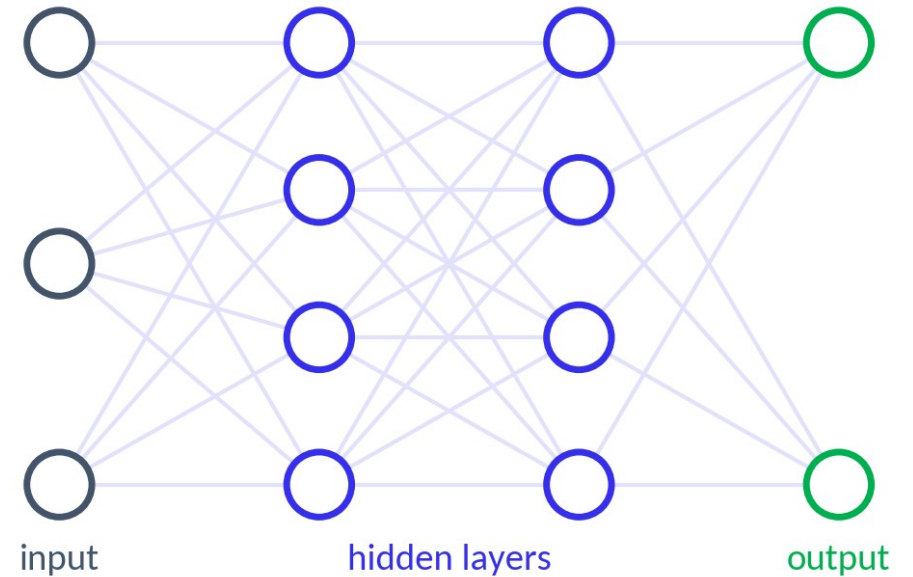
Parameters are learned (weights); **hyperparameters** are chosen outside learning (width, learning rate).

Representation: the form in which the input reaches the model.

Architecture: the form of the computation; it fixes the model class.

Learning algorithm: data to parameters. Applying the fitted model is prediction.

blue: mathematical objects **green**: learned from data **red**: chosen by us



The architecture is chosen; training chooses the weights. Logistic regression is one linear map and a sigmoid; a Transformer uses the same parameters at every position.

Basic **AI**/**ML**/**DL** Terminology

Generative and discriminative models

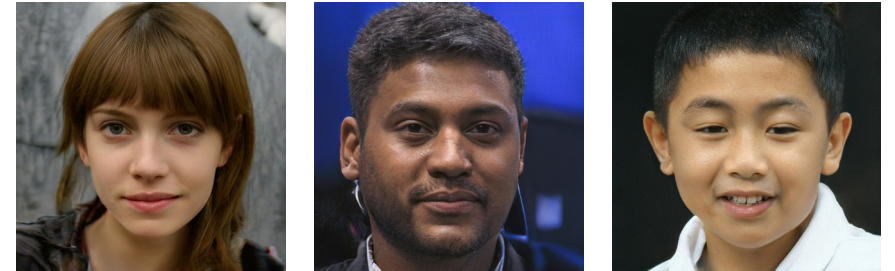
Discriminative: model the label given the input, and nothing else.

Generative: model how the inputs arise in each class, then classify with Bayes' rule.

Generative model: a probability distribution over the objects themselves, fitted to data. Generating means sampling; a conditional model samples given a prompt. A language model is a generative model of text.

generate: $x \sim q_\theta$, given a prompt c : $x \sim q_\theta(\cdot \mid c)$

text: $q_\theta(x_1 \cdots x_T) = \prod_t q_\theta(x_t \mid x_{<t})$



Sampling from a model of images gives new faces; sampling from a language model, token by token, gives new text.

Both classify. Only a generative model can produce new inputs.

Basic **AI/ML/DL** Terminology

Losses, risk, and generalization

Loss: the cost of one prediction. Log loss: minus the log probability of what happened.

Risk: the average loss on new examples. We can only compute it on the training set: the **empirical risk**.

Overfitting: the training loss falls while the loss on new examples rises.

Training, validation, and test data: fit, choose, and estimate: three separate jobs.

training: fit the parameters

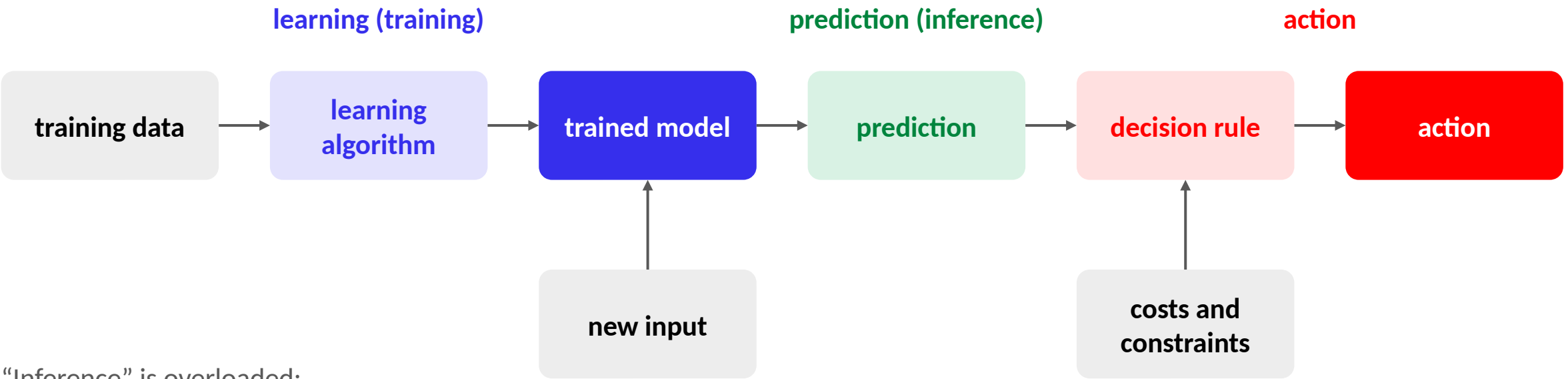
validation: make the choices

test: estimate performance, once

The separation is about information flow: a test set used to choose a model has become part of development. For text, split by document and remove duplicates.

Basic AI/ML/DL Terminology

Learning, prediction, and action



“Inference” is overloaded:

Term	Meaning	Example
statistical inference	reasoning from data to unknown quantities, with their uncertainty	a confidence interval for a model's accuracy
probabilistic inference	computing a quantity within a specified model	the probability of spam given an email's words
inference (deployment)	running a trained model	generating a response to a prompt

Basic **AI**/**ML**/**DL** Terminology

Example: a spam filter

practical problem	help a user manage unwanted email
abstract task	binary classification
learning paradigm	supervised, from labeled emails
learning setting	offline training on a collected dataset
input and target	features of an email; label spam or not
model family	logistic regression: a probability of spam
training objective	average log loss (binary cross-entropy), maybe regularized
training algorithm	gradient descent on that objective
trained model	the fitted weights and the rule they define
prediction	the probability that a new email is spam
action	a decision rule flags or routes the email

$$q_w(\text{spam} \mid x) = \sigma(w^\top x), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Lecture 1: (Artificial) Intelligence, Learning, Language, and Meaning

1. Short timeline highlighting language and its modeling, course description: what we will study and build
2. Basic AI/ML/DL terminology
3. Understanding
 - Intelligence
 - Learning
 - Language
 - Meaning
4. Hands-on: an autograd engine

Understanding Intelligence

Turing's question

1637 Descartes: a machine could never arrange words to reply sensibly to whatever is said, as even the dumbest person can.

1843 Lovelace: the Analytical Engine “has no pretensions to originate anything”; it does only what it is ordered to do.

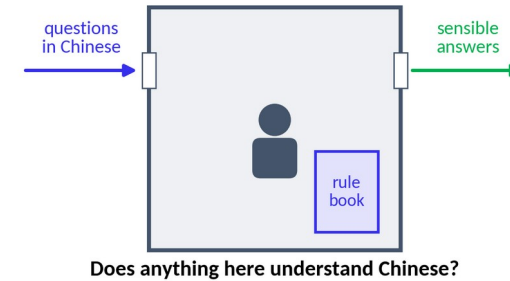
1949 Jefferson: no machine equals the brain until it writes a sonnet from thoughts and emotions felt. The question reaches the newspapers.

1950 Turing: “Can machines think?” is too ill-defined; replace it with a game. Build a child machine, and educate it.

What lasts is the method: replace “does it understand?” by “what does it compute, and how well, on a specified distribution of tasks?”

1966 ELIZA: a few pattern-matching rules left some users sure that it understood them. As a definition of intelligence, imitation is poor.

1980 The Chinese room: the attack from the other side: even flawless behavior, Searle argues, would not show understanding.



John Searle

A person with a rulebook answers Chinese questions sensibly without understanding Chinese. Reply: the whole system is the candidate. A language model is a learned procedure with no single part playing the person; the analogy sharpens the question of §7: can meaning be learned from form alone?

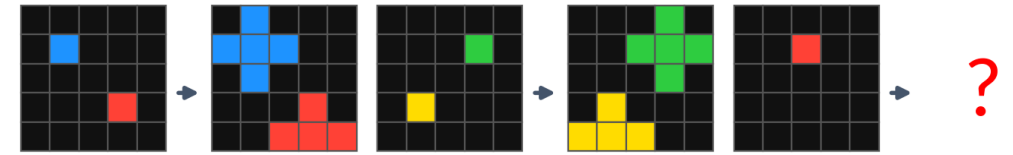
Understanding Intelligence

Definitions and measures

Russell and Norvig: thinking or acting, measured against humans or against an ideal of rationality.

Legg and Hutter: universal intelligence: expected performance across all computable environments, the simple ones weighted most.

Chollet: skill is not intelligence, since enough data buys skill.
Measure skill-acquisition efficiency.



two examples, then a new input: what is the rule?

an ARC-style puzzle: two examples, then a new input

A language model trained on much of the written record may show great skill on a benchmark; that says little about how efficiently it learns.

Understanding Intelligence

Symbols, networks, and the Dartmouth conjecture

1955 Dartmouth: every feature of intelligence can in principle be described so precisely that a machine can simulate it.

The symbolic program: intelligence is manipulating symbols by rules (Newell and Simon): theorem provers, planners, expert systems, grammars.

The connectionist program: simple units adjusting their connections: McCulloch and Pitts (1943), Rosenblatt's perceptron (1958).

1969 Minsky and Papert: one layer cannot compute parity or connectedness; the program stalled until backpropagation.

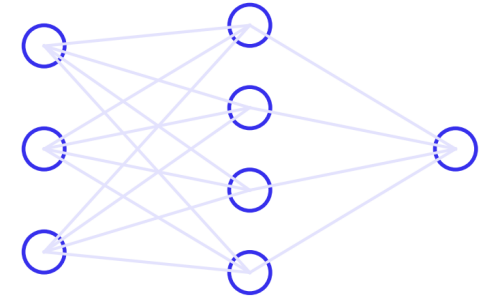
A language model is connectionist, yet it reads and writes symbols. Does it manipulate symbols in Newell and Simon's sense? Open.

symbols and rules

```
IF bird(x) AND NOT penguin(x)  
THEN flies(x)
```

```
bird(tweety) ⊢ flies(tweety)
```

units and learned weights



A network of simple units whose connections are learned, operating on discrete symbols, and prompted to carry out symbolic procedures, sometimes reliably and sometimes not.

Understanding Intelligence

Search and learning: Deep Blue and AlphaGo

1997 Deep Blue: deep search, with an evaluation built by hand from thousands of features; 480 custom chips. Its knowledge came from people, its strength from search.

2016 AlphaGo: about 250 legal moves per position, and no good hand-written evaluation. A policy network and a value network, learned from expert games and self-play, combined by tree search.

Both search; they differ in where the knowledge comes from: written down by people, or learned from data. The same division runs through this course.

	AlphaGo	a language model
learned policy	which move to try	which token comes next (pretraining)
improved by reward	self-play	post-training with rewards (Weeks 6–7)
search at play time	tree search	sample, verify, search (Week 8)

A pretrained language model is a learned policy over next tokens; rewards improve it as self-play improved AlphaGo's; test-time computation is search around a learned model.

Understanding Intelligence

The bitter lesson

2019 Sutton: across seventy years of AI, general methods that exploit computation, search and learning, eventually beat methods that build in human knowledge. Empirical, not a theorem; structure still matters.

2025 Sutton: language models learn from text that humans wrote, not from their own experience; systems that learn from experience will overtake them.



Richard Sutton on the Dwarkesh Podcast (2025)

Keep this in mind in Weeks 6 and 7, where language models do learn from rewards for their own outputs.

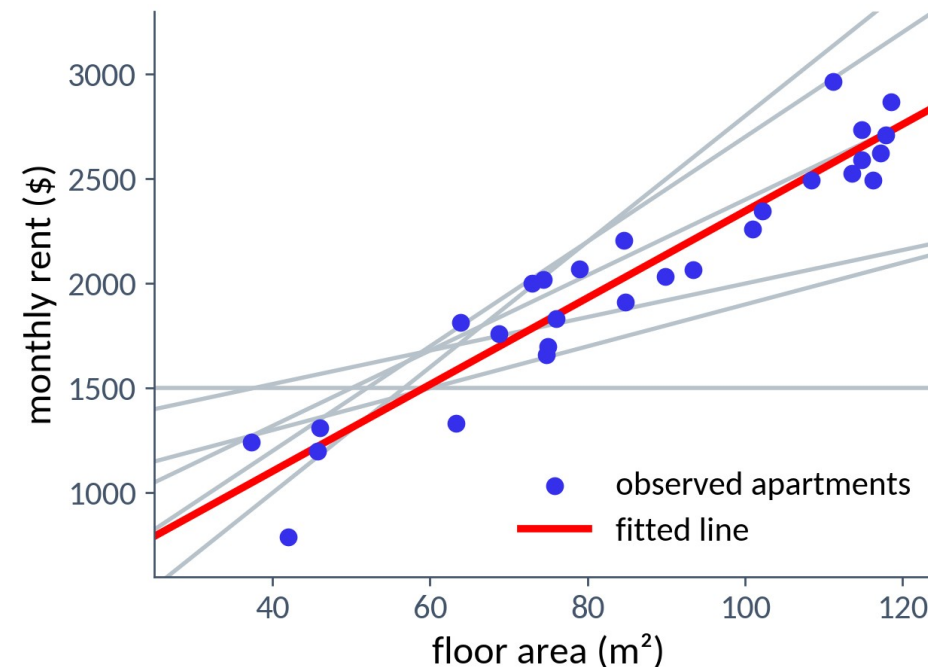
Understanding Learning

The problem of induction

1748 Hume: no argument from experience shows that the unseen will resemble the seen, without assuming the regularity it sets out to prove.

No free lunch: if every labeling is equally likely, every learner has expected error $1/2$ on the unseen points.

Inductive bias: learning works because real functions are simple: smooth, compositional. A learner succeeds by preferring them.



In a language model the inductive bias lives in the tokenizer, the architecture, the optimizer, the order of the data, and the objective.

Understanding Learning

Learning as a computational problem

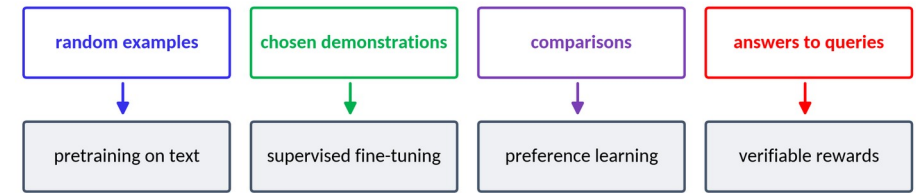
1984 Valiant: PAC learning. From m examples of an unknown target in a known class, output a hypothesis with small error, with high probability, in polynomial time.

The distribution is the assumption that the future resembles the past; **the class** is the inductive bias.

The Occam bound: a learner that fits every example needs only about the number of bits that name a hypothesis.

Time can bind: automata are hard to learn from random examples and easy with queries (Angluin's L^*). What you can learn depends on how you ask.

how the learner gets information



Pretraining supplies random examples; fine-tuning, chosen demonstrations; preference learning, comparisons; verifiable rewards, something like answers to queries.

Understanding Learning

From the perceptron to deep learning

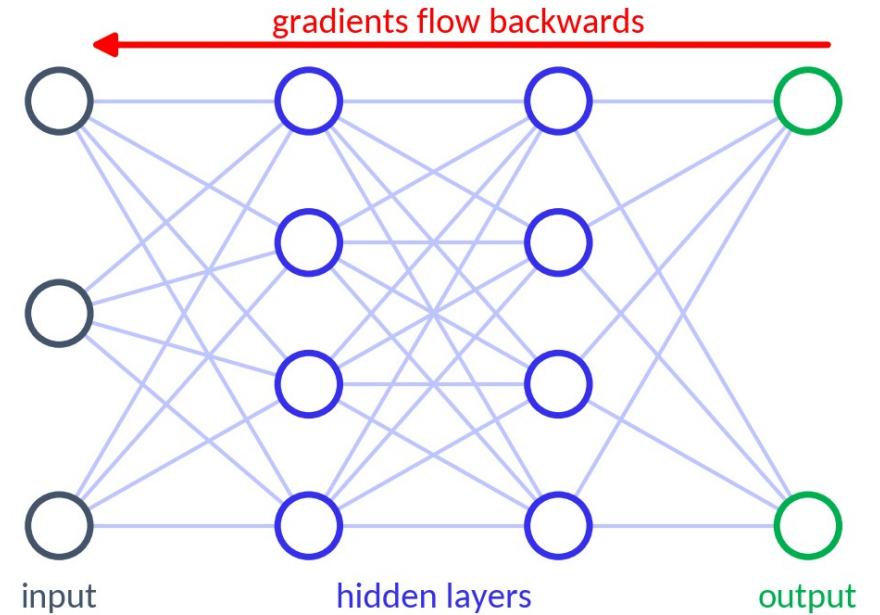
The perceptron: classify by the sign of a weighted sum; on each mistake, add the example to the weights.

Novikoff (1962): if a margin γ separates the data and inputs have norm at most R , it makes at most $(R/\gamma)^2$ mistakes, whatever the dimension.

One unit draws one line: exclusive or is out of reach.

Stack units: two hidden units each draw a line; the output unit weighs their votes.

Backpropagation: hidden units have no targets. Make every unit differentiable and follow the gradient: the chain rule assigns the blame.



Rumelhart, Hinton, and Williams (1986)

Replace the hard threshold by a smooth function such as the sigmoid; the gradient of the loss tells each hidden unit which way to move.

Understanding Learning

Neural networks and gradient descent

A layer: an affine map, then a fixed nonlinearity. One wide hidden layer approximates any continuous function (Cybenko, Hornik).

Expressivity is not learnability: the theorem says nothing about size, data, or whether gradient descent finds it.

Parameter sharing: same filter everywhere (convolutions), same update each step (recurrence), same parameters at every position (attention).

Gradient descent: step against the gradient; stochastic gradient descent uses a random minibatch.

One step: improves the prediction on the example it was computed from. Other examples: only held-out data can tell.

$$w = (0, 0), \quad x = (2, 1), \quad y = 1$$

$$\sigma(w^\top x) = 0.5, \quad \text{loss} = \ln 2 = 0.693$$

$$\nabla = (\sigma(w^\top x) - y) x = -\frac{1}{2} (2, 1)$$

$$w \leftarrow w - 0.1 \nabla = (0.1, 0.05)$$

$$\sigma(0.25) = 0.562 : \text{better on this example}$$

$$x' = (-2, 1), \quad y' = 1 : \text{loss } 0.693 \rightarrow 0.771$$

Logistic regression with log loss: the gradient on one example is the prediction error times the input.

Understanding Learning

Prediction is compression

Log loss is code length: a symbol predicted with probability p can be encoded in about $-\log_2 p$ bits (arithmetic coding). Conversely, every prefix code is a probability model (Kraft).

Cross-entropy: coding a source p with a model q costs $H(p) + \text{KL}(p||q)$ bits per symbol. A better predictor is a better compressor; the gap to the best code is exactly the KL divergence.

Training is compression: maximum likelihood minimizes the KL divergence from the data distribution, so a language model trained on next-token log loss is a compressor for text. It runs the other way too: gzip can generate text ([GziPT](#)).

Simplicity, briefly: naming a hypothesis costs bits too. Minimum description length: prefer the hypothesis whose own description plus the data coded with it is shortest.

Learning is prediction, and prediction is compression: good prediction, short description, and simplicity are three views of one idea.

$$\min_H \left[\underbrace{L(H)}_{\text{bits to name } H} + \underbrace{-\log_2 q_H(D)}_{\text{bits for the data, coded with } H} \right]$$

$$M(x) = \sum_{p: U(p) \text{ prints } x} 2^{-|p|}$$

A hypothesis earns its complexity only by compressing the data. Solomonoff's mixture over programs cannot be computed, but on any computable source its total error is bounded by the source's description length (§A.4).

Understanding Language

What a sentence is

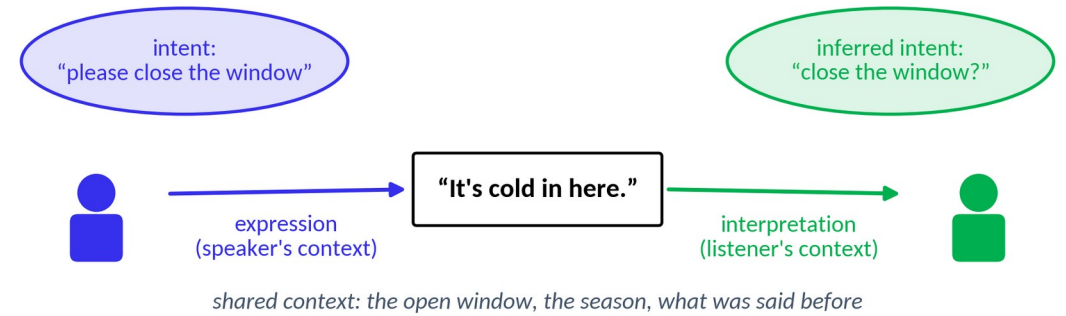
A sequence of marks: symbols on a page or sounds in time.

A grammatical construction: words grouped into phrases by rules.

An expression about something: it refers, and can be true or false.

A move in an interaction: it asks, tells, promises, or requests.

This section: structure and statistics. Meaning and use come next.



Use and intent: \$7.

Understanding Language

Rules and productivity

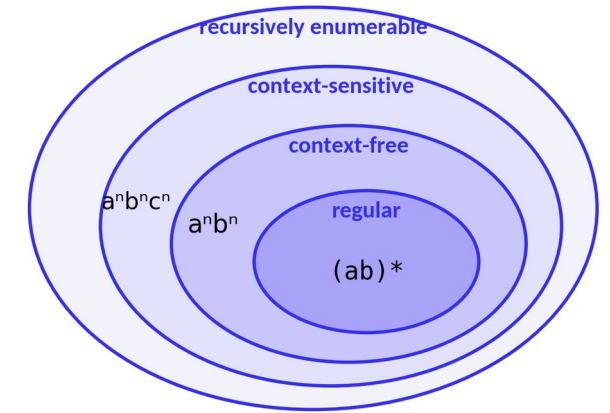
Pāṇini's Aṣṭādhyāyī: about four thousand rules that derive the forms of Sanskrit; the oldest grammar that survives complete.

Productivity: a finite description determines unboundedly many expressions, including ones never observed.

1956 Chomsky: a grammar is a finite device that generates exactly the grammatical sentences; the hierarchy classifies such devices.



Noam Chomsky



Three Models for the Description of Language (1956). The course takes the statistical route (Weeks 2 and 3).

Understanding Language

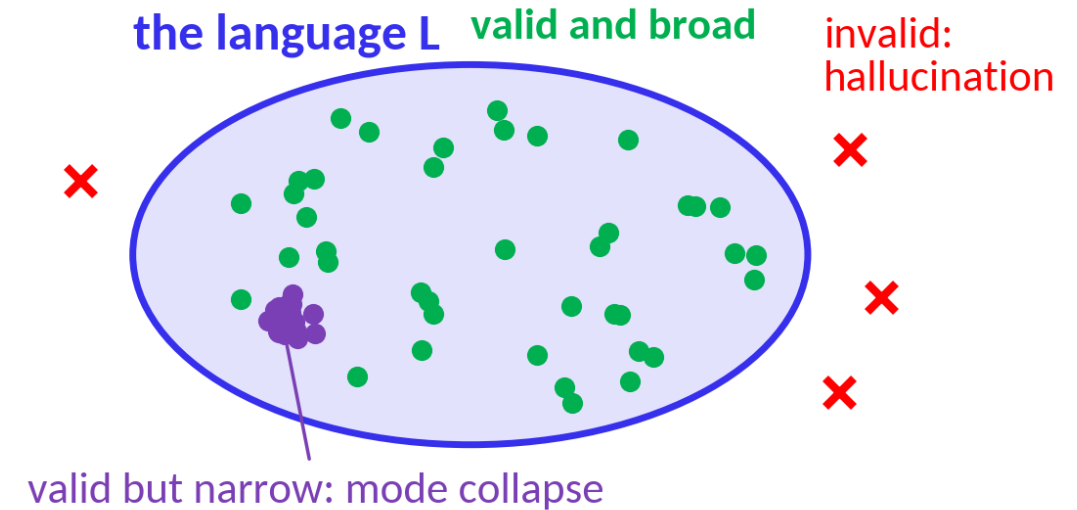
Learning a language from examples

1967 Gold: shown a language's strings in any order, no learner identifies even the regular languages: it cannot detect overgeneralization.

1980 Angluin: exactly which classes are learnable from positive examples (tell-tale sets). The poverty of the stimulus (Chomsky).

1969 Horning: from random samples, probabilistic context-free grammars are learnable: frequencies are evidence.

2024 Kleinberg and Mullainathan: generating new valid strings is possible for every countable class. The catch: validity against breadth.



Hallucination against mode collapse: for next-token predictors, being both valid and broad is impossible for most classes (Kalavasis, Mehrotra, Velegkas 2025), unless negative examples are supplied; Kleinberg and Wei (2025) keep a positive fraction.

Understanding Language

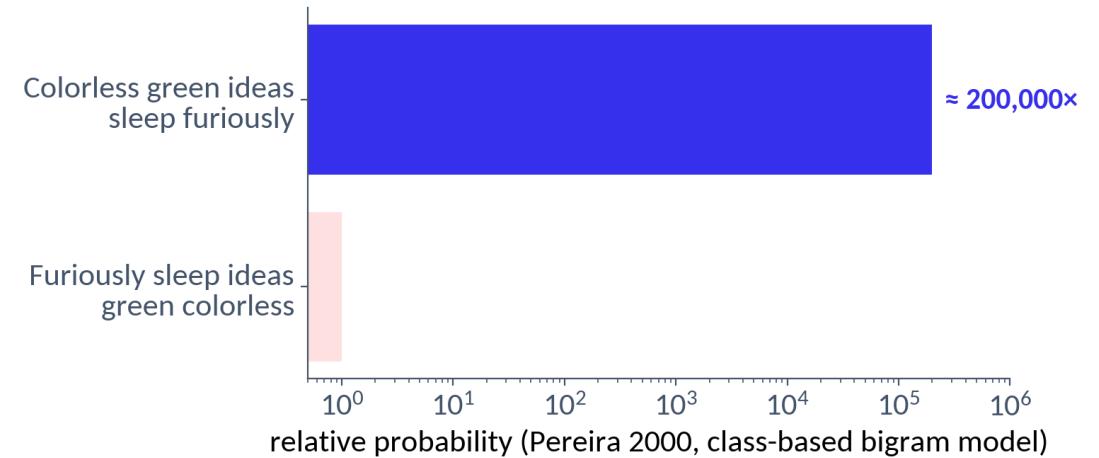
Prediction and statistics

1948 Shannon: English as a stochastic source: letters, then letters given the previous ones, then words given the previous word.

1951 the guessing game: people guessing the next letter: about one bit per letter with long context (0.6 to 1.3).

1957 Chomsky: “Colorless green ideas sleep furiously” and its reversal were both unseen, so statistics must treat them alike?

2000 Pereira: a class-based bigram model prefers the grammatical one by about 200,000 times.



Sharing statistical strength across similar words assigns sensible probabilities to unseen sentences: the central idea of Weeks 2 and 3. Norvig (2011), On Chomsky and the Two Cultures of Statistical Learning.

Understanding Language

The company a word keeps

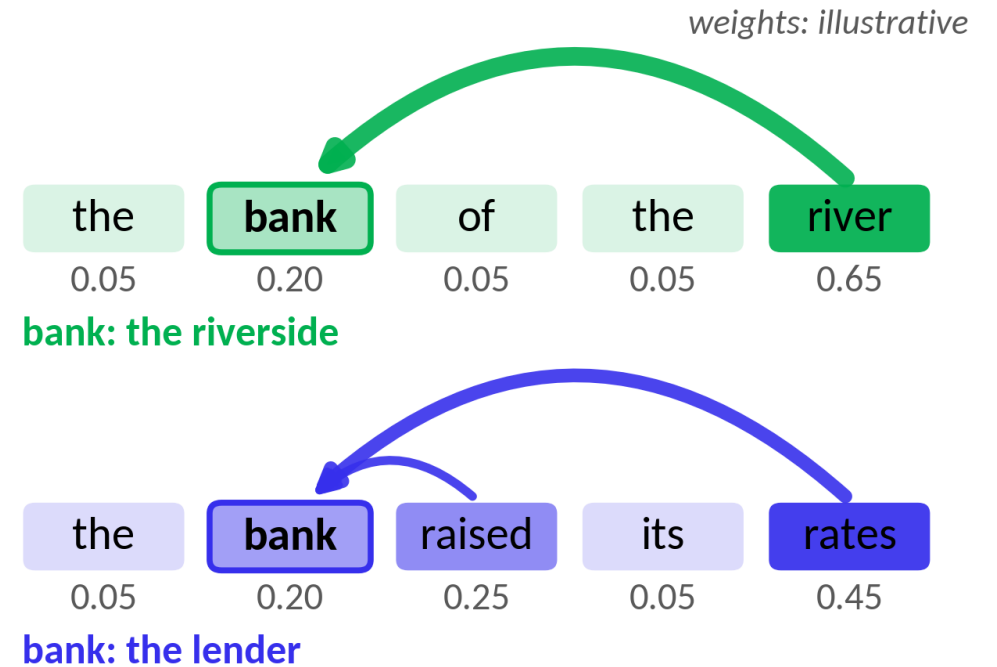
1954 Harris: words that occur in similar environments are similar: differences of meaning correspond roughly to differences of environment.

Six sentences: without knowing any meanings, one sees that *oculist* and *eye-doctor* share every environment, and *lawyer* shares half.

1957 Firth: “You shall know a word by the company it keeps.” Word vectors represent a word by statistics of its contexts; neural models learn them together with the predictor (Bengio et al., 2003).

Attention: one vector per word is too coarse: “bank” keeps different company in “the bank of the river” and “the bank raised its rates”. Attention gives each occurrence its own vector, a weighted mix of the words around it (Week 3).

A language model trained to predict the next token is a learned summary of the company every word keeps.



The weights are computed from the words themselves. A word is known, quite literally, by the company it keeps in the sentence at hand.

Understanding Meaning

What words are about

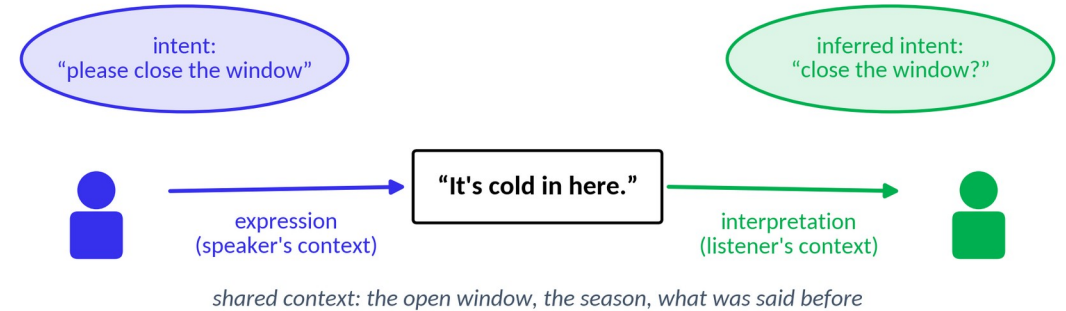
Reference: an expression picks out something in the world. “The morning star” and “the evening star” both refer to Venus, with different senses (Frege, 1892).

Composition: the meaning of a whole depends on its parts and how they combine: “Every student read a paper” has two readings.

Use: for many words the meaning is the use (Wittgenstein, 1953), and speakers communicate more than they say (Grice, 1975).

Intent: a speaker chooses an utterance to convey an intent; the listener reconstructs one from the words and the context they share.

A language model sees only text: no label for the referent or the intent is ever supplied. The text carries evidence about both. Is it enough?



“It's cold in here” can report the temperature, explain a coat, or ask for the window to be closed. The words fix much less than the meaning.

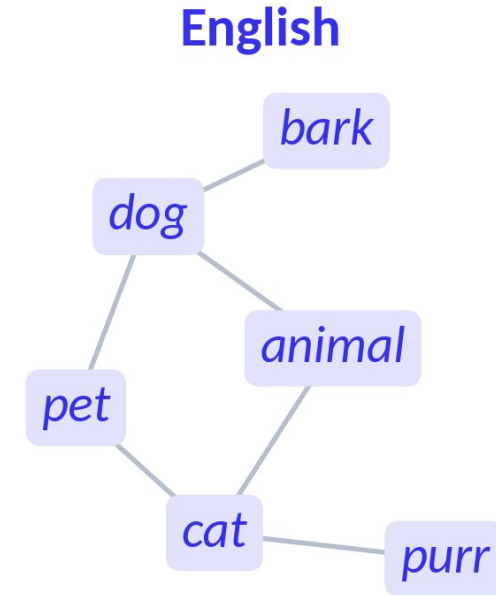
Understanding Meaning

The grounding problem

2020 Bender and Koller: a hyperintelligent octopus taps the cable between two islanders, learns the statistical patterns of their messages, and impersonates one of them convincingly.

The bear: chased by a bear and holding two sticks, one islander asks for advice. The octopus cannot help: nothing in the patterns connects the words to sticks and bears.

1990 Harnad: the symbol grounding problem: how can the symbols of a formal system mean anything to the system itself, rather than only to the people who interpret it?



A dictionary defines every word by other words. A newcomer who knows no English learns nothing from it alone: every definition leads only to more words.

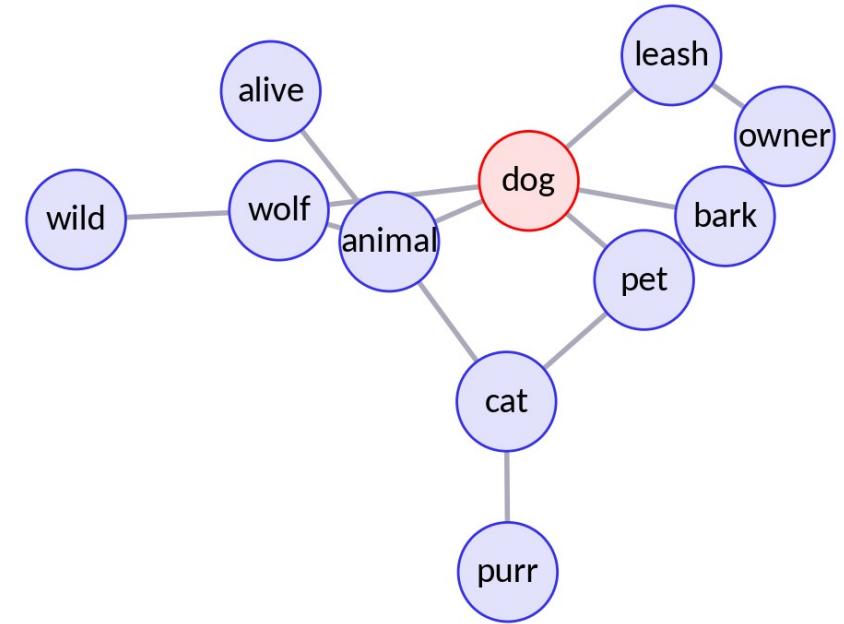
Understanding Meaning

Meaning up to isomorphism

A second dictionary: if the newcomer also has a dictionary of their own language, and the two networks of cross-references have nearly the same shape, structure alone matches word to word.

Word vectors: vectors learned from each language's text separately have similar geometry: a linear map, learned from a small seed dictionary or from none, translates many words.

2022 Piantadosi and Hill: the meaning of a concept lies largely in its relations to other concepts, its conceptual role, which a model trained on text can learn.



Not settled, and this course does not settle it.

The precise question: what about the world and about intent can be recovered from the distribution of text, and what do demonstrations, comparisons, and checks add?

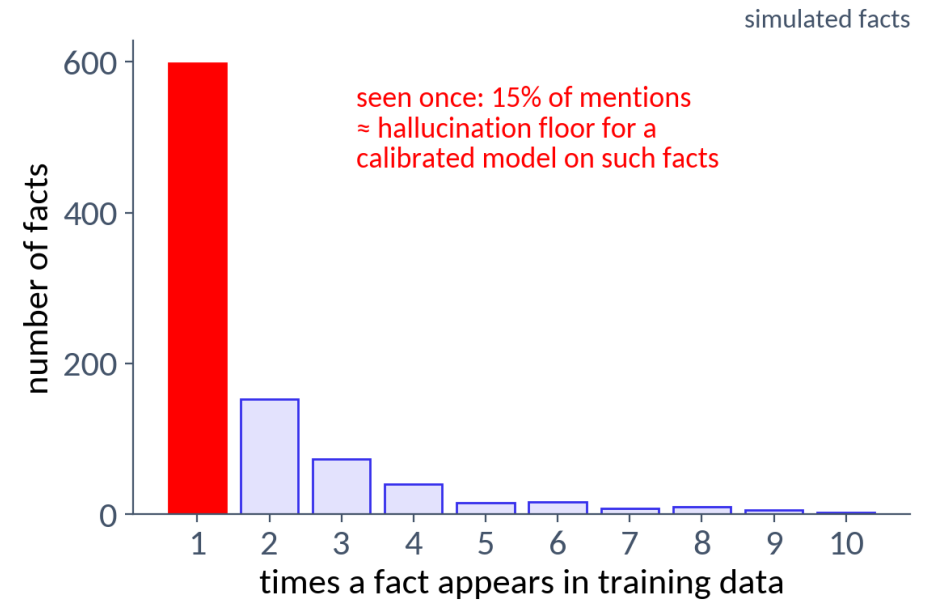
Understanding Meaning

Probable is not true

1948 Shannon: the semantic aspects of communication are “irrelevant to the engineering problem.” A language model inherits the stance: it is rewarded for probable text, true or not.

2024 Kalai and Vempala: for arbitrary facts, whose truth no pattern reveals, a calibrated model must generate false ones at about the rate of facts seen exactly once in training.

The scope: a lower bound for arbitrary facts only: no statistical reason to hallucinate facts seen many times or that follow patterns. To go below it, give up some calibration (post-training, Week 7) or consult a source.



Simulated facts. The fraction of facts seen once is the Good–Turing estimate of the probability of the unseen.

Lecture 1: (Artificial) Intelligence, Learning, Language, and Meaning

1. Short timeline highlighting language and its modeling,
course description: what we will study and build
2. Basic AI/ML/DL terminology
3. Understanding
 - Intelligence
 - Learning
 - Language
 - Meaning
4. Hands-on: an autograd engine

Hands-on: an Autograd Engine

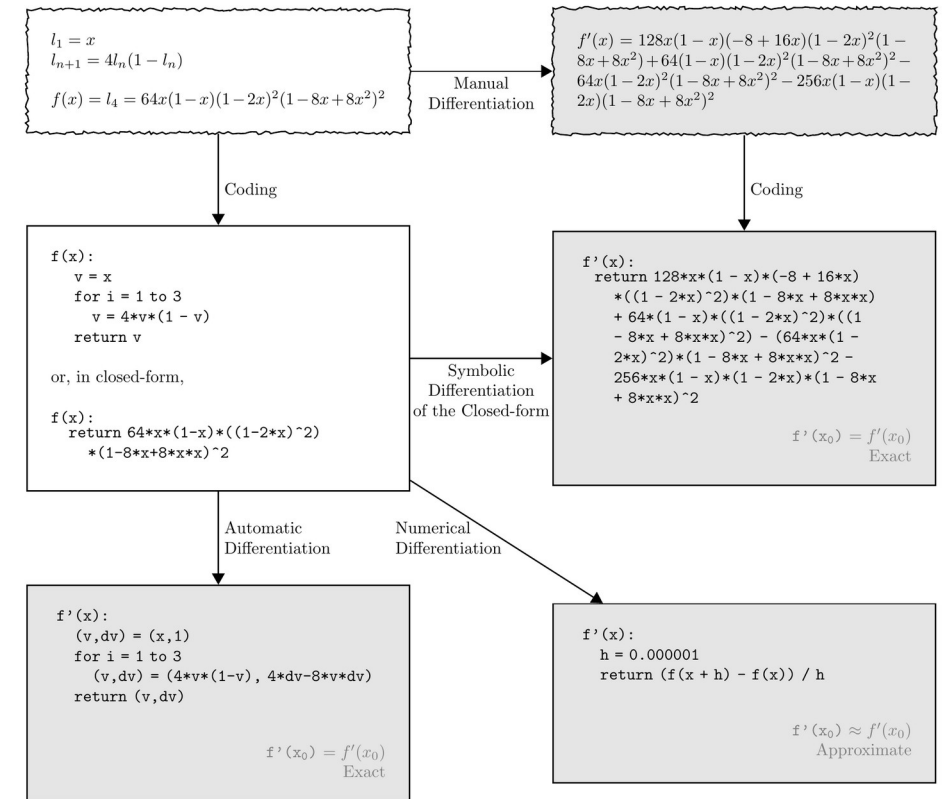
Three ways to compute a derivative

A small program: the logistic map, run for three steps. How do we get its derivative? (Figure 17 of the notes, from Baydin et al., 2018.)

Symbolic: expand the program into one formula, then differentiate. Exact, but a value used twice is copied twice.

Numerical: evaluate the program at nearby points. Approximate, and one input direction at a time.

Automatic: apply each operation's exact derivative to the values the program computes. Exact up to rounding, a constant factor more work, and it follows the loops and branches the program took.



The right-hand column of the figure: each value v is carried together with its derivative dv .

Against numerical differentiation, autodiff wins twice: **accuracy** (the smaller win) and **cost** (the decisive one).

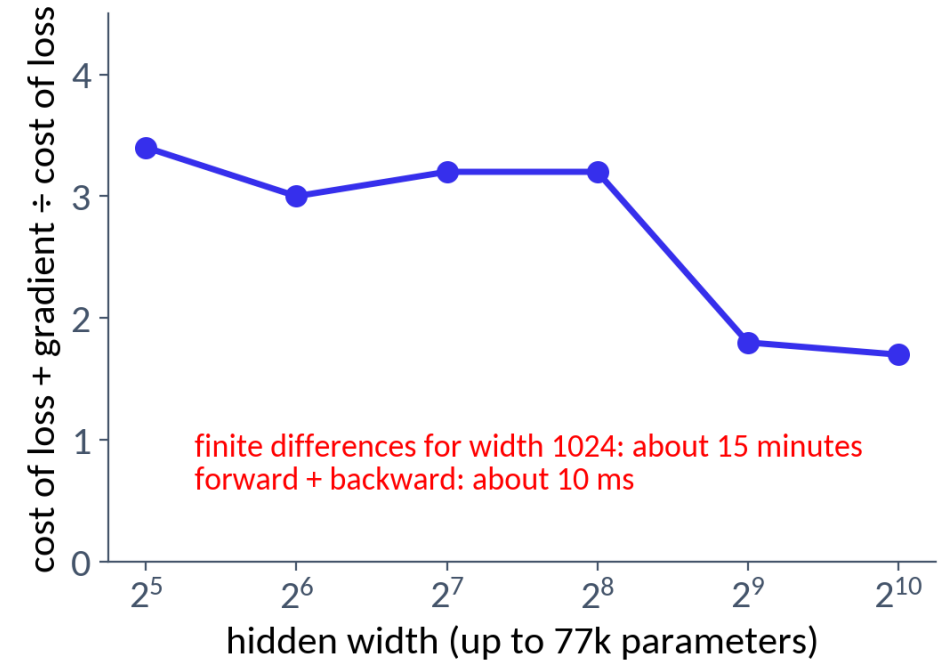
Hands-on: an Autograd Engine

Why reverse mode

Forward mode: carries the derivative with respect to one input forward, alongside the values: n inputs, n passes, like finite differences.

Reverse mode: starts from the single output and sends its sensitivity back to every input at once. The whole gradient costs a small constant factor more than the function (Baur and Strassen, 1983). The price is memory.

At scale: 100 million parameters: 200 million evaluations for finite differences, about 2 or 3 for forward plus backward.



In the notebook, width 1024: about 15 minutes for finite differences against about ten milliseconds for forward plus backward. This, more than the digits, is why billion-parameter networks can be trained.

Hands-on: an Autograd Engine

Tensors and computation graphs

A tensor: holds its data, a gradient of the same shape, its parents, and a backward rule.

Every operation records these: so evaluating a loss builds a graph from the leaves, the inputs and parameters, to the loss.

```
X = Tensor(rng.normal(size=(4, 3)))
W = Tensor(rng.normal(size=(3, 2)),
            requires_grad=True)
b = Tensor(np.array([0.5, -0.5]),
            requires_grad=True)

Z = X @ W + b          # (4,3) @ (3,2) + (2,) -> (4,2)
L = (Z ** 2).mean()    # one number
```

```
Z: shape (4, 2)  op +   parents: @, leaf
L: shape ()      op /   parents: sum, leaf
L = 3.913
```

Part A of the Week 1 notebook, section 1. The engine computes a mean as a sum divided by a count.

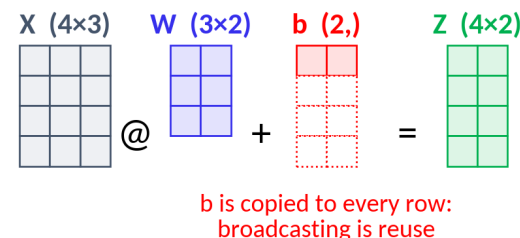
Hands-on: an Autograd Engine

The backward pass

One local rule per operation: a vector–Jacobian product.
Visit the nodes in reverse topological order.

A value used twice: the contributions from its uses add:
the multivariable chain rule.

Broadcasting is reuse: b is added to every row, so its
gradient sums the rows. Every gradient has the shape of its
tensor.



backward

$$\frac{\partial L}{\partial W} = X^T G \quad (3 \times 2)$$

$$\frac{\partial L}{\partial b} = \sum_{\text{rows}} G \quad (2,)$$

$$\frac{\partial L}{\partial X} = G W^T \quad (4 \times 3)$$

gradient shape = tensor shape

$$\frac{\partial L}{\partial W} = X^T G, \quad \frac{\partial L}{\partial X} = G W^T, \quad \frac{\partial L}{\partial b} = \sum_{\text{rows}} G$$

A useful check when deriving rules by hand: the gradient has the
shape of the tensor it belongs to.

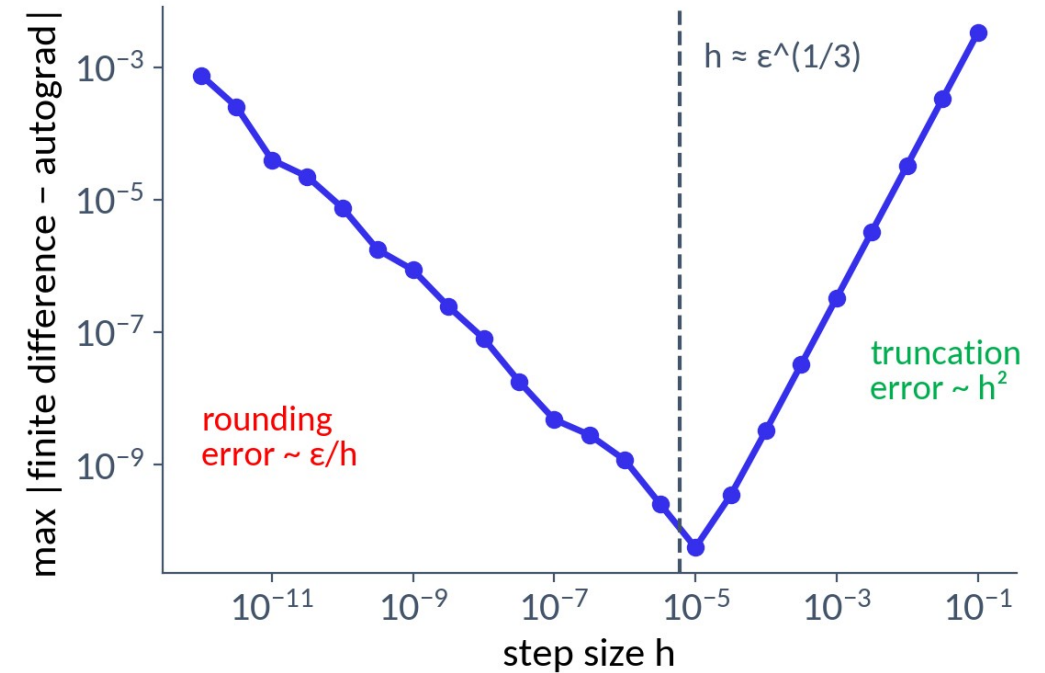
Hands-on: an Autograd Engine

Checking gradients

A wrong rule does not crash: it silently gives a wrong gradient, and training quietly does worse. Check every rule against the central difference.

Two errors: truncation of order h^2 , rounding of order ϵ/h : the best h is near 10^{-5} in double precision.

And against PyTorch: the engine's tests compare its gradients with PyTorch's.



Error of the central difference against the step size, for a small network.

Hands-on: an Autograd Engine

Try it: the [Week 1 notebook](#)

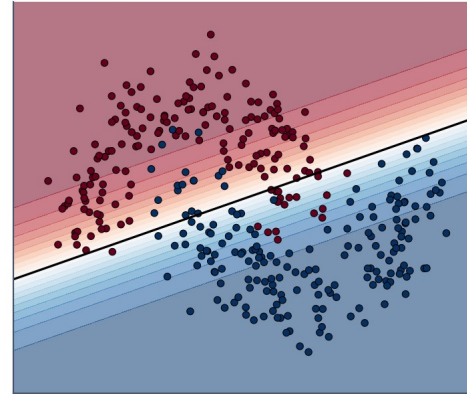
Get the code: `tinylm`, the course's codebase, built from scratch.

Part A: a tensor, the graph walked backwards, gradient shapes, a derivative checked, one gradient step.

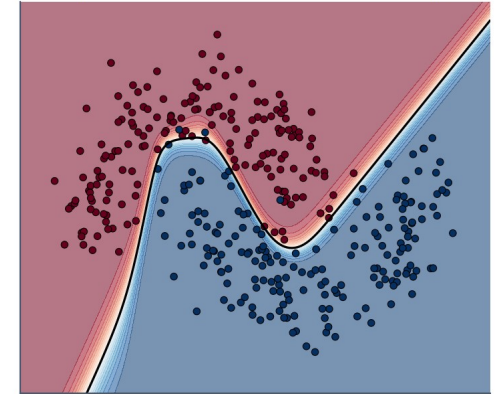
Part B: the cost of gradients, then logistic regression and a two-layer network trained with the engine.

Exercise, five minutes: let $L = \|Xv\|^2$. Work out $\partial L / \partial v$ by hand and compare it with `v.grad`; then change the shape of `b` and predict `b.grad`.

logistic regression: validation accuracy 0.85



two-layer network: validation accuracy 0.97



Two interleaved half-moons: logistic regression draws a straight boundary; one hidden layer of tanh units bends it.

Hands-on: an Autograd Engine

From here to language models

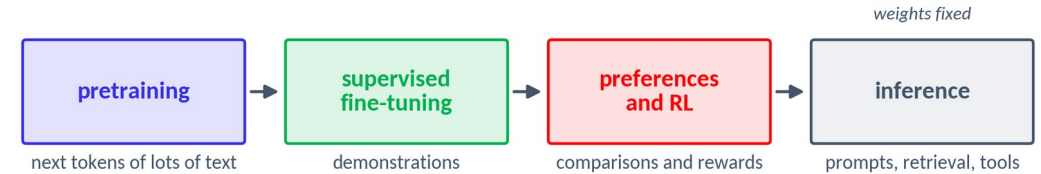
Each conditional: is a classifier over the vocabulary, computed by a network from the prefix.

Training: minimizes the log loss of the observed next tokens: self-supervised.

Generation: samples one token at a time and feeds it back as input.

Four stages: each with its own source of learning signal.

Learning mid-conversation is conditioning: a fact in the context changes the predictions, and nothing persists once the context is gone.

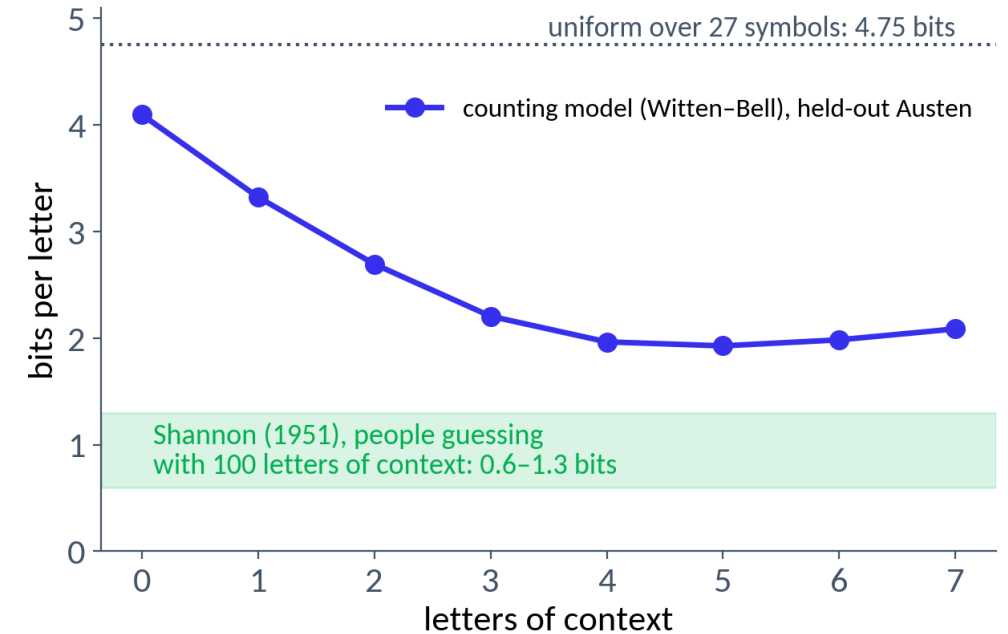


Pretraining (Weeks 2–4), supervised fine-tuning (5, 7), preferences and reinforcement learning (6, 7), inference (5, 8). Only inference leaves the parameters unchanged.

Hands-on: an Autograd Engine

Next week: from counts to neural networks

1. How do we assign probabilities to strings?
2. How do we measure whether a predictor is any good?
3. Why does counting run out, and what replaces it?



Word vectors and neural language models.